



Universidad Latinoamericana de Ciencia y Tecnología
Pruebas de Aseguramiento de la Seguridad del Software

Proyecto Final

Avance 1

La Voz Silenciosa

Profesor:

Clarence Ricketts Torres

Estudiantes:

Karla Calderón Sánchez

Lorenzo Corredor Ramírez

Jose Solorzano Cruz

Valeria Rojas Morales

José Mauricio Mendez Bermúdez

21 de Junio, 2026

II Cuatrimestre

La Voz Silenciosa

1. Integrantes del equipo

Rol Scrum	Responsable	Responsabilidad Principal
Product Owner	Karla Calderón Sánchez	Define y prioriza el backlog; representa al cliente y a los usuarios; valida que el producto entregue valor; gestiona el alcance del proyecto; responsable del Acta Constitutiva y de la coordinación/ensamblaje del entregable final.
Scrum Master	José Mauricio Méndez Bermúdez	Facilita el proceso Scrum; elimina impedimentos; protege al equipo de interferencias externas; asegura que se sigan las prácticas ágiles; responsable del Backlog en Jira y de la Arquitectura/Estándares.
Tech Lead	Valeria Rojas	Construye los entregables del proyecto; auto-organiza el trabajo del sprint; responsable de la Propuesta Inicial (Fase A).
Developer Team	Lorenzo Corredor Ramírez / Jose Solorzano	Construye los entregables del proyecto; responsable de la WBS/EDT (3.2).

2. Documentación Fundamental

Nombre del proyecto: La Voz Silenciosa.

Patrocinador (Sponsor): Consorcio de universidades "Red Universitaria de Bienestar Estudiantil de Costa Rica" (RUBE-CR), representado por su Dirección de Bienestar Estudiantil. El patrocinador autoriza el proyecto, aporta los recursos y aprueba los hitos de cada entrega.

Gerente del Proyecto (PM): José Mauricio Méndez Bermúdez. Coordina la planificación general, el cronograma de los siete sprints y la comunicación con el patrocinador.

2.1. Acta Constitutiva del Proyecto

El presente documento oficializa la existencia del proyecto "La Voz Silenciosa", autoriza el inicio del trabajo y establece la visión, el alcance y los criterios de éxito que guiarán al equipo durante el desarrollo de la plataforma.

Dueño del Alcance

Karla Calderón Sánchez (Product Owner)

Responsable de definir, priorizar y aprobar los requerimientos del proyecto, así como de gestionar el alcance y representar las necesidades de los usuarios.

Propuesta Inicial

Desarrollar una plataforma web y móvil (PWA) llamada La Voz Silenciosa que permita a estudiantes universitarios realizar autoevaluaciones de salud mental, acceder a recursos de apoyo, encontrar servicios de orientación y solicitar ayuda de manera confidencial.

Problema

Muchos estudiantes universitarios enfrentan problemas de salud mental, altos niveles de estrés y desconocen los servicios de apoyo disponibles. Actualmente no existe una plataforma centralizada que facilite la detección temprana y el acceso a estos recursos.

Empresa / Organización

Consortio de universidades RUBE-CR Instituciones universitarias que buscan mejorar el acceso a recursos de apoyo emocional y bienestar estudiantil mediante una solución tecnológica común.

Objetivos del Proyecto (SMART)

Objetivo general:

- Desarrollar e implementar la plataforma La Voz Silenciosa, accesible como PWA, que permita a los estudiantes del consorcio RUBE-CR realizar autoevaluaciones de salud mental, acceder a recursos de apoyo y solicitar ayuda confidencial, completando las siete épicas del proyecto conforme a la Ley 8968 a lo largo del II Cuatrimestre 2026 (siete sprints).

Objetivos específicos:

- Garantizar la confidencialidad de los datos de salud, de modo que las pruebas de Row Level Security cierren con cero accesos cruzados entre usuarios antes del cierre del Sprint 7.
- Habilitar la derivación oportuna, conectando cada resultado de autoevaluación con el recurso correspondiente según nivel de riesgo, con el botón de ayuda operativo y su protocolo de respaldo probado antes del cierre del Sprint 3.
- Entregar las siete épicas marcadas como "Must" completadas según su Definición de Terminado al cierre del Sprint 7, con la auditoría de la Ley 8968 sin hallazgos críticos pendientes.

Alcance, Exclusiones y Criterios de Éxito

El proyecto entrega una plataforma web instalable como app (PWA) con autoevaluación validada, directorio de servicios RUBE-CR, motor de derivación según nivel de riesgo, botón de ayuda con líneas reales, grupos de apoyo con foro confidencial, panel administrativo y la documentación de cierre. Todo bajo el marco de la Ley 8968.

Queda fuera de alcance: el diagnóstico clínico (la autoevaluación orienta, no diagnostica); la atención psicológica directa dentro de la plataforma (la herramienta deriva hacia profesionales y líneas reales, no los sustituye), la integración con

expedientes académicos o sistemas internos de cada universidad, las aplicaciones nativas para tiendas (iOS/Android), ya que la entrega es una PWA.

El proyecto se considera exitoso si cumple, como mínimo: el MVP de autoevaluación y directorio desplegado y funcional al cierre del Sprint 2, el botón de ayuda operativo con líneas verificadas y protocolo de respaldo probado antes del cierre del Sprint 3, cero accesos cruzados entre usuarios en las pruebas de Row Level Security, las siete épicas marcadas como "Must" completadas según su Definición de Terminado, y la auditoría de cumplimiento de la Ley 8968 cerrada sin hallazgos críticos pendientes.

Justificación del Proyecto

La inversión en el proyecto "La Voz Silenciosa" se justifica por la magnitud y urgencia de la crisis de salud mental que afecta a la población universitaria en Costa Rica, así como por la ausencia de una herramienta digital estructurada que articule la detección temprana, la información de servicios y la derivación efectiva hacia recursos de apoyo. La población universitaria presenta niveles de malestar psicológico por encima de la media nacional, y una proporción importante del estudiantado desconoce los servicios de apoyo que ofrece su propia institución. Una plataforma común permitirá a las universidades del consorcio RUBE-CR responder de forma ágil, confidencial y basada en datos, reduciendo la brecha entre la necesidad del estudiantado y la capacidad institucional de respuesta, generando capacidades reutilizables que pueden escalarse a las demás instituciones miembro.

Gobernanza del Proyecto

El equipo Scrum está conformado por Karla Calderón como Product Owner, responsable de definir y priorizar el backlog, representar al cliente y gestionar el alcance del proyecto; José Mauricio Méndez Bermúdez como Scrum Master, responsable de facilitar el proceso Scrum, eliminar impedimentos y asegurar las prácticas ágiles; y los Developers Valeria Rojas, Lorenzo Corredor y Jose Solorzano, responsables de construir los entregables del proyecto. El Product Owner tiene autoridad para definir prioridades, aceptar o rechazar entregables y gestionar la visión del producto, mientras que el Scrum Master facilita las ceremonias Scrum (Sprint Planning, Daily Scrum, Sprint Review y Sprint Retrospective) y protege al equipo de interferencias externas. El

proyecto se desarrolla en sprints de dos semanas durante el II Cuatrimestre 2026, con entregas incrementales que inician con el MVP de autoevaluación y directorio, seguidos por recursos de emergencia, grupos de apoyo, panel administrativo y aplicación móvil, finalizando con pruebas y cierre.

Planificación, Restricciones, Riesgos y Supuestos

Los entregables clave se distribuyen en ocho sprints: Sprint 0 de preparación (semanas 1-2), Sprint 1 y 2 para el MVP de autoevaluación y directorio (semanas 3-6), Sprint 3 para recursos de emergencia (semanas 7-8), Sprint 4 para grupos de apoyo (semanas 9-10), Sprint 5 para panel administrativo (semanas 11-12), Sprint 6 para aplicación móvil (semanas 13-14) y Sprint 7 para pruebas y cierre (semanas 15-16). El proyecto opera bajo restricciones de tiempo (debe completarse dentro del cuatrimestre), recursos (equipo estudiantil con disponibilidad parcial y presupuesto limitado) y legales (cumplimiento de la Ley 8968 de protección de datos). Se han identificado riesgos como el manejo inadecuado de datos sensibles (mitigado con RLS y cifrado), fallos en el botón de ayuda (mitigado con protocolos de respaldo y pruebas exhaustivas), sobre-alcance (mitigado con backlog priorizado y enfoque en MVP), baja participación de estudiantes (mitigado con coordinación con departamentos de bienestar) y disponibilidad parcial del equipo (mitigado con horarios claros y planificación realista).

El proyecto parte de varios supuestos. Las universidades del consorcio RUBE-CR facilitan su directorio de servicios actualizado a tiempo para cargarlo. El área de psicología revisa y valida el instrumento de autoevaluación (PHQ-9/GAD-7) dentro del cronograma. Las líneas de emergencia que alimentan el botón de ayuda están disponibles y verificadas durante toda la operación. El equipo conserva su disponibilidad parcial sin bajas durante el cuatrimestre. Y los servicios gratuitos de Supabase, Vercel/Netlify y GitHub Actions mantienen las cuotas necesarias para un equipo de cinco. Si alguno de estos supuestos no se cumple, el riesgo asociado pasa a tratarse como impedimento en el Daily.

Bloque de firmas

Rol	Nombre	Firma	Fecha
-----	--------	-------	-------

Nivel 2

- 1.1 Gestión y Documentación
- 1.2 MVP: Autoevaluación y Directorio
- 1.3 Recursos de Emergencia
- 1.4 Grupos de Apoyo
- 1.5 Panel Administrativo
- 1.6 Aplicación Móvil (PWA)
- 1.7 Pruebas y Cierre

Nivel 3

- 1.1.1 Acta, WBS y Backlog en Jira
- 1.1.2 Repositorio, CI y proyecto base
- 1.1.3 Plan de Arquitectura y Estándares
- 1.2.1 Cuestionario de Autoevaluación
- 1.2.2 Directorio de Servicios RUBE-CR
- 1.2.3 Motor de Derivación
- 1.2.4 Registro, sesión y seguridad base
- 1.3.1 Botón de Ayuda Inmediata
- 1.3.2 Directorio de Líneas de Emergencia
- 1.3.3 Protocolo de Activación y Respaldo
- 1.4.1 Gestión de Grupos de Apoyo
- 1.4.2 Calendario y Notificaciones
- 1.4.3 Foro de Interacción Confidencial
- 1.5.1 Panel de Control para Administradores
- 1.5.2 Dashboard de Indicadores
- 1.5.3 Gestión de Usuarios y Roles
- 1.6.1 Instalación como App (PWA)
- 1.6.2 Notificaciones
- 1.7.1 Plan y Ejecución de Pruebas
- 1.7.2 Auditoría de Cumplimiento (Ley 8968)
- 1.7.3 Documentación y Cierre

Nivel 4 (Paquetes de Trabajo)

- 1.1.1.1 Acta Constitutiva aprobada
- 1.1.1.2 WBS/EDT aprobada
- 1.1.1.3 Proyecto Jira con épicas e historias
- 1.1.2.1 Repositorio GitHub con main protegida (/docs y /web)
- 1.1.2.2 Pipeline de CI (lint + pruebas en cada PR)
- 1.1.2.3 Proyecto base React + Vite + TS + Supabase
- 1.1.3.1 Documento de arquitectura y stack
- 1.1.3.2 Estándares de código y Definición de Terminado
- 1.2.1.1 Instrumento validado (PHQ-9/GAD-7) revisado por psicología
- 1.2.1.2 Edge Function de cálculo de puntaje
- 1.2.1.3 Pantalla de resultado y nivel de riesgo
- 1.2.2.1 Catálogo de servicios por universidad
- 1.2.2.2 Buscador y filtros (modalidad, horario)
- 1.2.3.1 Reglas de derivación según nivel de riesgo
- 1.2.3.2 Recomendación de recurso al estudiante
- 1.2.4.1 Registro/login con correo institucional y seudónimo
- 1.2.4.2 Esquema de datos con RLS y cifrado base
- 1.2.4.3 Shell PWA instalable y accesible (WCAG AA)
- 1.3.1.1 Botón de ayuda visible 24/7 con líneas verificadas
- 1.3.1.2 Registro de la acción sin exponer datos sensibles
- 1.3.2.1 Listado de líneas de emergencia verificadas
- 1.3.3.1 Diseño del flujo de emergencia
- 1.3.3.2 Protocolo de respaldo (fallback) ante fallo del canal
- 1.4.1.1 Creación e inscripción a grupos temáticos
- 1.4.2.1 Calendario de sesiones
- 1.4.2.2 Avisos de actividades
- 1.4.3.1 Foro en tiempo real con seudónimo (Supabase Realtime)
- 1.4.3.2 Moderación de contenido de riesgo
- 1.5.1.1 Gestión del directorio y las derivaciones
- 1.5.1.2 Mantenimiento del instrumento de autoevaluación
- 1.5.2.1 Tablero de indicadores agregados y anónimos
- 1.5.3.1 Roles (estudiante, administrador) con RLS
- 1.5.3.2 Configuración de políticas RLS por usuario
- 1.6.1.1 Manifiesto e instalación en el teléfono

- 1.6.1.2 Service worker y caché básico (modo offline)
- 1.6.2.1 Web Push y correo
- 1.7.1.1 Pruebas unitarias de lógica crítica (Vitest)
- 1.7.1.2 Pruebas E2E de flujos críticos (Playwright)
- 1.7.1.3 Pruebas de seguridad (RLS, OWASP, ZAP)
- 1.7.2.1 Lista de verificación de privacidad
- 1.7.2.2 Corrección de hallazgos e informe
- 1.7.3.1 Documentación técnica y manual de usuario
- 1.7.3.2 Acta de cierre

Explicación de la EDT

El EDT descompone el proyecto en cuatro niveles. El nivel 1 es el proyecto entero. El nivel 2 son los siete entregables mayores, uno por cada épica del backlog, la gestión y documentación, el MVP de autoevaluación y directorio, los recursos de emergencia, los grupos de apoyo, el panel administrativo, la aplicación móvil como PWA, y las pruebas y el cierre. El nivel 3 abre cada entregable en sus componentes, y el nivel 4 son los paquetes de trabajo, que son las tareas que los desarrolladores realizan. Existe un enfoque en entregables, no por actividades sueltas, así cada caja produce un resultado que se puede revisar y dar por terminado.

3.2 Backlog del Proyecto

El backlog se gestiona en Jira, a cargo del Scrum Master, como ya adelantan el Acta Constitutiva y la sección de arquitectura. Se eligió Jira por encima de una hoja de cálculo o un tablero genérico por tres razones concretas: maneja de forma nativa la jerarquía épica → historia → subtarea necesaria para colgar cada historia de su entregable; estima en story points y muestra el avance real del sprint con burndown y velocity; y su nivel gratuito alcanza de sobra para un equipo de cinco. Como alternativa se valoró GitHub Projects, que integra issues y commits directamente con el repositorio, pero Jira da mejor soporte a las ceremonias Scrum que pide el curso y a la trazabilidad con la WBS.

El proyecto se crea en Jira como proyecto de tipo Scrum con la clave LVS (La Voz Silenciosa). La jerarquía sigue tres niveles que reflejan la WBS:

- **Épicas:** una por cada entregable mayor de la WBS (Nivel 2). Son siete.

- **Historias de usuario:** derivan de los componentes de Nivel 3 y 4, escritas en formato “Como... quiero... para...”.
- **Subtareas:** los paquetes de trabajo técnicos de cada historia (por ejemplo, configurar RLS o escribir la Edge Function del puntaje).

Cada historia se etiqueta con su código de WBS y se asigna a un sprint, de modo que el tablero y la WBS cuentan la misma historia.

Épicas (derivadas del Nivel 2 de la WBS)

Épica	Entregable mayor	WBS	Sprint(s)	Prioridad
EP-01	Gestión y Documentación	1.1	Sprint 0	Must
EP-02	MVP: Autoevaluación y Directorio	1.2	Sprint 1–2	Must
EP-03	Recursos de Emergencia	1.3	Sprint 3	Must
EP-04	Grupos de Apoyo	1.4	Sprint 4	Should
EP-05	Panel Administrativo	1.5	Sprint 5	Should
EP-06	Aplicación Móvil (PWA)	1.6	Sprint 6	Should
EP-07	Pruebas y Cierre	1.7	Sprint 7	Must

Historias de usuario (derivadas del Nivel 3 y 4)

Se presenta el conjunto representativo que cubre todos los entregables de la WBS. La estimación usa story points en escala Fibonacci (SP).

ID	Historia de usuario	WBS	Sprint	MoSCoW	SP
HU-01	Como Scrum Master quiero el proyecto creado en Jira con tablero Scrum, épicas e historias cargadas para gestionar el avance con trazabilidad a la WBS	1.1.1	S0	Must	3
HU-02	Como equipo quiero el repositorio en GitHub con main protegida y CI en cada PR para	1.1.2	S0	Must	5

	evitar que un cambio roto llegue a la rama principal				
HU-03	Como desarrollador quiero el proyecto base (React + Vite + TS + Supabase) levantado con .env.example para empezar a construir sin fricción de configuración	1.1.2	S0	Must	3
HU-04	Como estudiante quiero completar una autoevaluación validada (PHQ-9 / GAD-7) de forma anónima para conocer mi estado emocional sin exponer mi identidad	1.2.1	S1	Must	8
HU-05	Como estudiante quiero recibir un resultado con mi nivel de riesgo y una recomendación para saber qué paso dar después	1.2.1	S1	Must	5
HU-06	Como estudiante quiero buscar y filtrar los servicios del directorio RUBE-CR por modalidad y horario para contactar el recurso más conveniente	1.2.2	S2	Must	5
HU-07	Como sistema quiero derivar automáticamente al recurso adecuado según el puntaje para conectar al estudiante con el apoyo correcto	1.2.3	S2	Must	5
HU-08	Como estudiante quiero registrarme con correo institucional y seudónimo para acceder de forma segura sin revelar quién soy	1.2.4	S1	Must	5
HU-09	Como estudiante en crisis quiero un botón de ayuda visible 24/7 que conecte con una línea real para recibir apoyo humano inmediato	1.3.1	S3	Must	8

HU-10	Como sistema quiero un protocolo de respaldo si el canal principal del botón falla para no dejar sin atención una crisis	1.3.3	S3	Must	5
HU-11	Como estudiante quiero ver el directorio de líneas de emergencia para contactarlas directamente	1.3.2	S3	Should	3
HU-12	Como estudiante quiero unirme a un grupo de apoyo temático para compartir mi experiencia en un entorno seguro	1.4.1	S4	Should	5
HU-13	Como estudiante quiero participar en un foro confidencial en tiempo real con seudónimo para interactuar con pares sin exponerme	1.4.3	S4	Should	8
HU-14	Como estudiante quiero recibir avisos de sesiones y actividades del grupo para no perderme el acompañamiento	1.4.2	S4	Could	3
HU-15	Como administrador de bienestar quiero un panel para gestionar el directorio y las derivaciones para mantener la información al día	1.5.1	S5	Should	5
HU-16	Como orientador quiero un dashboard con indicadores agregados y anónimos para priorizar la atención con datos	1.5.2	S5	Should	5
HU-17	Como administrador quiero gestión de usuarios y roles con RLS para que cada quien acceda solo a lo que le corresponde	1.5.3	S5	Must	5
HU-18	Como estudiante quiero instalar la plataforma como app en mi teléfono (PWA) para acceder rápido desde el celular	1.6.1	S6	Should	5

HU-19	Como estudiante quiero recibir notificaciones push para enterarme de respuestas y recordatorios	1.6.2	S6	Could	3
HU-20	Como líder de QA quiero ejecutar el plan de pruebas (unitarias, e2e, seguridad y RLS) para garantizar que los flujos críticos no fallen	1.7.1	S7	Must	8
HU-21	Como equipo quiero auditar el cumplimiento de la Ley 8968 con una lista de verificación para confirmar el manejo correcto de los datos sensibles	1.7.2	S7	Must	5
HU-22	Como equipo quiero la documentación técnica y el acta de cierre para formalizar la entrega del proyecto	1.7.3	S7	Must	3

Priorización y estimación

Se prioriza el backlog con MoSCoW, combinado con el valor para el estudiante y el riesgo técnico. Lo que toca una crisis real —la autoevaluación, el botón de ayuda y las políticas de RLS— entra como Must y se estima y se prueba primero. Lo accesorio —notificaciones push, calendario de grupos— queda como Could y es lo primero que cede si el tiempo aprieta. La estimación usa story points en escala Fibonacci (1, 2, 3, 5, 8, 13): no se miden horas, se mide el esfuerzo relativo, y la velocidad del equipo se ajusta con lo que realmente cierra cada sprint.

Seguimiento del backlog

El seguimiento es el ciclo Scrum que fija el Acta. El backlog se refina antes de cada Planning; el Daily mueve las tarjetas por el tablero

(To Do → In Progress → In Review → Done);

El Review valida cada historia contra sus criterios de aceptación y la Definición de Terminado; y la Retrospective ajusta el proceso.

El burndown del sprint y la velocidad del equipo dan la lectura objetiva del avance. Una historia solo pasa a Done cuando cumple su DoD, no cuando “el código corre”.

Criterios de aceptación de las historias críticas

HU-04 — Autoevaluación anónima. El cuestionario corresponde a un instrumento validado (PHQ-9/GAD-7) revisado por el área de psicología; se completa sin nombre real; el puntaje se calcula en una Edge Function de Supabase, no en el navegador; y el resultado nunca se comparte con terceros.

HU-09 — Botón de ayuda 24/7. El botón está visible en toda la plataforma; conecta con una línea o persona real, nunca con un bot; si el canal principal falla, se activa el protocolo de respaldo (HU-10); y la acción queda registrada para seguimiento sin exponer datos sensibles.

HU-17 — Roles y RLS. Las políticas de Row Level Security impiden que un usuario autenticado lea o modifique datos de otro; se prueba explícitamente el intento de acceso cruzado; y ninguna regla sensible queda expuesta en el cliente.

Trazabilidad con la WBS y el Acta

El Scrum Master mantiene la coherencia entre los tres documentos: cada paquete de trabajo de la WBS tiene su historia en el backlog, cada historia se rastrea a un entregable definido en el Acta, y los nombres de módulos y roles se escriben igual en los tres. Esa trazabilidad forma parte del aseguramiento de calidad del proyecto y es lo que permite verificar, en cualquier momento, que el Acta, la WBS y el backlog hablan del mismo proyecto.

3.3 Arquitectura y Estándares

3.3.1 Arquitectura de la solución

La plataforma se construye como una sola aplicación web responsiva que también se instala como app en el teléfono (PWA). Con esto cubrimos el acceso desde computadora y desde celular con un mismo código, sin mantener productos separados ni un sistema de sincronización entre ellos.

El frontend se comunica directamente con Supabase, que funciona como backend administrado y aporta la base de datos PostgreSQL, la autenticación, el almacenamiento de archivos y las suscripciones en tiempo real. La lógica que debe correr del lado del servidor, como el cálculo del puntaje de la autoevaluación o el disparo de una alerta,

vive en funciones serverless de Supabase para no exponer reglas sensibles en el navegador.

Los datos sensibles se protegen en tres capas: cifrado en tránsito por HTTPS, cifrado en reposo en la base, y Row Level Security para que cada usuario solo lea y escriba lo suyo. RLS es la pieza central del modelo: aunque alguien alcance la API, las políticas de la base impiden que vea registros de otra persona.

3.3.2 Tecnologías y lenguajes

Capa	Elección	Por qué
Frontend / PWA	React + Vite + TypeScript	Curva conocida, se instala en el teléfono, un solo código para web y móvil
Estilos / UI	Tailwind CSS	Interfaz consistente y rápida de construir
Backend (BaaS)	Supabase	Postgres, Auth, RLS y Storage sin montar un servidor propio
Base de datos	PostgreSQL (en Supabase)	RLS y cifrado nativos, ya previstos en la WBS
Autenticación	Supabase Auth	Manejo seguro de sesiones sin guardar contraseñas a mano
Tiempo real / foro	Supabase Realtime	Suscripciones para el foro confidencial y las notificaciones
Lógica de servidor	Supabase Edge Functions	Cálculo de puntaje y alertas fuera del navegador
Notificaciones	Web Push + correo	Sin depender de las

		tiendas de apps
Hosting	Vercel o Netlify (plan gratuito)	Despliegue automático desde GitHub
Integración continua	GitHub Actions	Linter y pruebas en cada Pull Request
Repositorio	GitHub	Central del código y la documentación

3.3.3 Uso de inteligencia artificial

Usamos IA como herramienta de desarrollo. Durante la construcción, el equipo se apoya en asistentes como Claude Code para tareas como la generación código repetitivo, escribir pruebas unitarias, explicar errores. Las decisiones de arquitectura, el diseño de la seguridad y la lógica del cálculo de riesgo las define el equipo.

Todo los resultados que produce el asistente pasa por las mismas reglas que el código hecho a mano, la revisa una persona en un Pull Request, corre el linter y las pruebas, y nadie hace merge de algo que no entiende o no comprende cómo agrega valor al código del proyecto. Ponemos cuidado extra en el código sensible (políticas RLS, autenticación, manejo de datos).

Nunca se pega en una herramienta de IA información real o sensible, como respuestas de estudiantes, credenciales o variables de entorno. Para trabajar con los agentes se utilizarán datos ficticios.

Por último, el producto en sí no usa IA para evaluar ni aconsejar a un estudiante y el puntaje sale de un instrumento validado con reglas fijas, y el botón de ayuda conecta con personas reales.

3.3.4 Estándares de desarrollo

- **Lenguaje único:** TypeScript en el frontend y en las funciones de servidor.

- **Estilo de código:** ESLint y Prettier en el repositorio; el formato se valida antes de cada merge.
- **Commits:** convención Conventional Commits (feat, fix, docs) para un historial legible.
- **Ramas:** main protegida; cada funcionalidad va en su rama y entra por Pull Request con al menos una revisión.
- **Privacidad (Ley 8968):** consentimiento explícito antes de recolectar datos, recolección mínima, derecho de eliminación y separación de los datos de salud del resto.
- **Accesibilidad:** WCAG 2.1 AA en lo básico (contraste, navegación por teclado, lectores de pantalla). Pesa más que de costumbre acá, porque quien usa la plataforma puede estar en un mal momento.
- **Anonimato:** la autoevaluación se completa sin nombre real; las cuentas usan seudónimo y el correo institucional solo verifica que la persona pertenece a la universidad.

3.3.5 Metodología

El equipo trabaja con Scrum en sprints de dos semanas, como ya fija el Acta. Cada sprint cierra con una versión funcional desplegada. Las ceremonias son Planning, Daily, Review y Retrospective. Una historia se da por terminada cuando cumple la Definición de Terminado, no cuando "el código corre".

Repositorio en la nube

El proyecto vive en un repositorio de GitHub con dos carpetas: /docs para la documentación del curso y /web para el código. main está protegida y todo entra por Pull Request. Un flujo de GitHub Actions corre el linter y las pruebas en cada PR, así nada roto llega a main. El README explica cómo levantar el proyecto e incluye un .env.example con las variables necesarias, sin llaves reales. El backlog y los sprints se gestionan en Jira, a cargo del Scrum Master.

Enlace del repositorio: https://github.com/jsolorzanoc/voz_silenciosa.git

(Para validez del entregable se crea el repo con visibilidad pública, una vez se empieza a desarrollar este pasará a privado, dándole acceso al equipo de desarrollo y al profesor para validar el código en la entrega final)

Enlace del Backlog :

El backlog se gestiona en Jira, en un proyecto Scrum con la clave LVS, a cargo del Scrum Master. La estructura refleja la WBS en tres niveles, siete épicas, una por cada entregable mayor, veintidós historias de usuario en formato y cuarenta y tres subtarefas que corresponden a los paquetes de trabajo del cuarto nivel. Cada historia se estima en story points con escala Fibonacci, se prioriza con MoSCoW y se asigna a uno de los siete sprints del cuatrimestre. Cada tarjeta lleva una etiqueta con su código de WBS, así el Acta, la WBS y el tablero hablan del mismo proyecto.

Tablero del proyecto:

<https://lavozsilenciosa.atlassian.net/jira/software/projects/LVS/boards/2/timeline?atlOrigin=eyJpIjoiZTlhYTE4ZDZmYzRmNDczN2E1MGRkNDAwM2ViOTQzOGEiLCJwIjoiaaiJ9>

Aseguramiento de calidad y seguridad (QA)

Pruebas funcionales

- Unitarias con Vitest para la lógica de cálculo (puntaje de riesgo, reglas de derivación).
- Extremo a extremo con Playwright para los flujos que no pueden fallar: registro y login, completar la autoevaluación, y el botón de ayuda.

Pruebas de seguridad

- Dependencias: npm audit y Dependabot para librerías con vulnerabilidades conocidas.
- Análisis estático: ESLint con reglas de seguridad y, si da el tiempo, Semgrep.
- Análisis dinámico: OWASP ZAP contra el ambiente de pruebas.
- Pruebas de RLS: confirmar que un usuario autenticado no puede leer ni modificar datos de otro. Es el caso crítico de un sistema con datos de salud.
- Revisión del OWASP Top 10 sobre autenticación y formularios: control de acceso, inyección, XSS.
- Secretos: todas las llaves en variables de entorno, nunca en el repositorio.

Privacidad (Ley 8968): consentimiento, minimización, cifrado en tránsito y en reposo, y opción de borrar la cuenta.

Definición de Terminado (DoD): una historia está terminada cuando cumple sus criterios de aceptación, pasa el linter y las pruebas automáticas, fue revisada en un PR, no agrega vulnerabilidades según npm audit, respeta las políticas de RLS y privacidad, y quedó desplegada en el ambiente de pruebas.

Coherencia documental: como parte del QA revisó que el Acta, la WBS y el backlog hablen del mismo proyecto. Cada paquete de trabajo de la WBS tiene su historia en el backlog, cada historia se rastrea a un entregable, y los nombres de módulos y roles se mantienen iguales en los tres documentos.